NIALL CASSIDY

DESIGN

A2 WJEC COMPUTER SCIENCE

NIALL CASSIDYNDIDATE NO: 0109
CENTER NO: 71741

Section 3: Design (15/100) marks

- **Specify, design and document screen layouts, reports and other forms of input and output required to create the user interface**

- **Design and document all data structures that will be required to produce the output for the solution to the problem together with the method of accessing the data in that data structure**

- **Ensure that all data entered into the system is valid.**

- **Design programming routines to be used to handle and process data within the proposed solution to achieve each objective. Document these designs using structured convention such as pseudo-code**

3.0 Design

Now that I've completed the analysis stage of the proposed solution and thoroughly investigated the existing system to identify the essential features required for the digital database; It's time to move on to the design phase of the system implementation. In this section, I will delve into each component of the system in detail, pointing out potential issues as they arise, and compiling valuable information to be used in subsequent stages of the project. This comprehensive analysis will employ various methods to clarify the relationships, processes, and other critical aspects of the final system.

3.1 Decomposition of problems into sub-problems

To assist in the design stage of this new software, I plan to break-down the large challenges associated with the system into smaller, more manageable issues to be tackled independently. Show below are the smaller features to be developed and implemented as a result of this process of decomposition.

Add:
Ensure the system has the capabilities to insert records into their chosen table.
Delete:
Ensure the system has the capabilities to delete records from their chosen table.
Edit:
Ensure the system has the capabilities to append/edit data in their chosen table.
Search:
Allows the system to verify if a certain record exists, ensuring only one instance of a certain record in the database.
View:
Allow the system to view the details of their chosen record from their chosen table.
Login:
Ensure that only approved users can access the system.

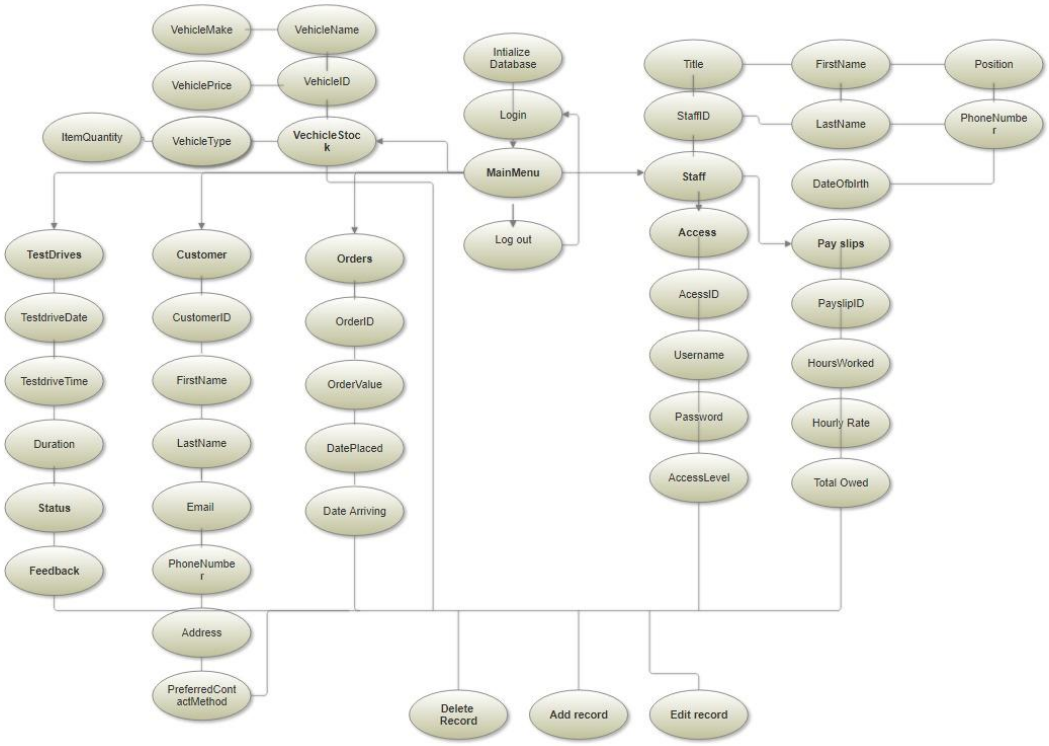
Determine level of access:

Ensure that only users with base level permissions have the corresponding access to certain files on the system.

Validation:
Ensure data on the system is appropriately and reasonably validated.

3.2 Visual Overview:

I think the best way to illustrate all the features of the new system is with a visual diagram. This diagram should encompass the main areas and sub-areas of the program, as well as the general operations and processes associated with each area of the new system to ensure no confusion and complete understanding of the system when viewing this diagram.



3.3 Database

An appropriate method for storing the necessary data is required for this approach. Considering the specifics of the proposed solution, a relational database is deemed the most suitable option. The database will be created using the 'SQLite3' module, as the solution will be implemented in Python. This database will consist of multiple tables, incorporating a significant portion of the data outlined in the previous overview.

As a relational database, it will utilise foreign keys to establish connections between tables. Accordingly, foreign keys will be implemented in each table within the database.

Test Drive Table

This table will hold data about test drive bookings and the corresponding details related to each test drive.

- **Test Drive ID** (Primary Key): A unique identifier for each test drive booking.
- **Customer ID** (Foreign Key referencing the **Customer** table): A reference to the customer who has booked the test drive.
- **Vehicle ID** (Foreign Key referencing the **VehicleStock** table): A reference to the vehicle being tested.
- **Test Drive Date**: The scheduled date for the test drive.
- **Test Drive Time**: The scheduled time for the test drive.
- **Duration**: Duration of the test drive (e.g., 30 minutes).
- **Status**: Status of the test drive (e.g., Pending, Completed, Cancelled).
- **Staff ID** (Foreign Key referencing the **Staff** table): The staff member handling the test drive.
- **Feedback**: A field for the customer to provide feedback after the test drive, if applicable.

Payslip Table

This table holds payroll details for each staff member.

- **Payslip ID**: Unique identifier for each payslip.
- **Staff ID**: Foreign Key linking to the **Staff** table, identifying which staff member the payslip belongs to.
- **Hours Worked**: Number of hours worked by the staff member during the pay period.
- **Hourly Rate**: The hourly rate for the staff member.
- **Total Owed**: The total amount to be paid based on the hours worked and hourly rate (calculated automatically).

Access Table

This table contains login credentials and access levels for staff members.

- **Access ID**: Unique identifier for each access record.
- **Username**: The staff member's username for system login.
- **Password**: The encrypted password for system security.
- **Access Level**: Defines the access rights (e.g., Admin, User, Manager), determining what parts of the system the staff member can access.

VehicleStock Table

The **Stock** table stores information about vehicles and parts in inventory.

- **VehicleID**: Unique identifier for each item or vehicle in Vehicle.
- **Vehicle Name**: Name of the item (e.g., "2019 Honda Civic").
- **Vehicle Model**: Manufacturer of the item (e.g., "Honda").
- **Vehicle Price**: Price of the item or vehicle.
- **Vehicle Type**: Type of vehicle stored (e.g Truck, Sporty, Luxary).
- **Item Quantity**: Quantity of the item available in stock.

Orders Table

This table holds details of orders placed for vehicles.

- **Order ID**: Unique identifier for each order.
- **Order Value**: Total value of the order (calculated as quantity × item price).
- **Date Placed**: Date when the order was placed.
- **Date Arriving**: Date when the order is expected to arrive in Vehicle.
- **Vehicle ID**: Foreign Key linking to the **VehicleStock** table, indicating which item is being ordered.

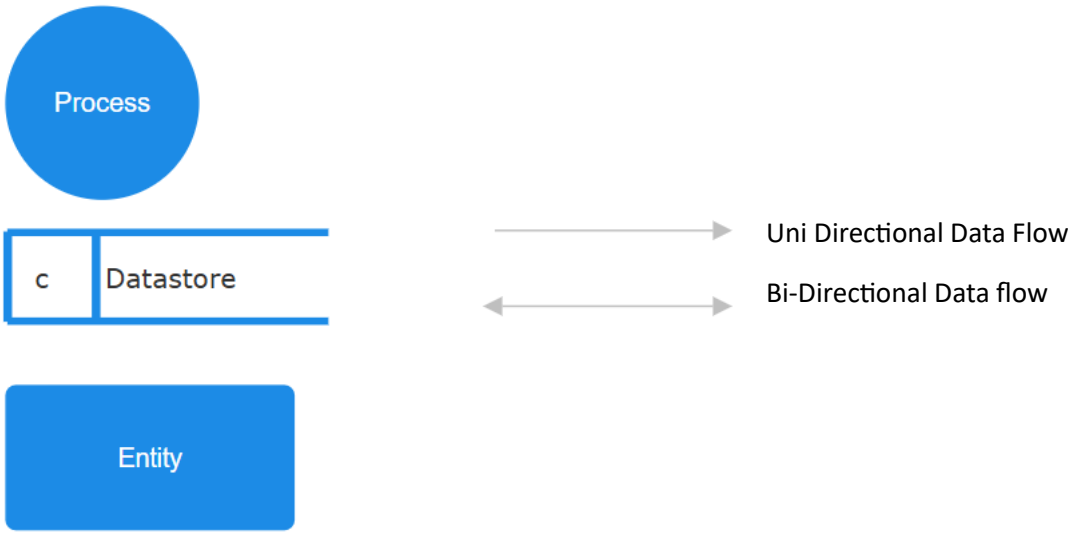
Customer Table

This table holds information about customers who are interested in buying, or have bought vehicles or services at Mulligan Motors.

- **Customer ID**: Unique identifier for each customer.
- **First Name**: First name of the customer.
- **Last Name**: Last name of the customer.
- **Email Address**: Email contact for the customer.
- **Phone Number**: Phone number for contacting the customer.
- **Address**: Customer's home address (could include street, city, and zip code).
- **Preferred Contact Method**: Email, phone, or other communication preference

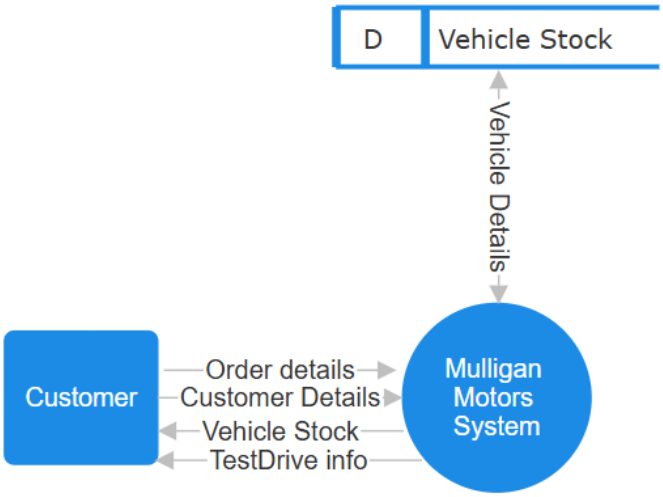
3.4 Data Flow Diagrams

With the following data flow diagram (DFD), I plan to display the flow of data in the new system, taking into account inputs, processes and outputs. The symbols and elements I will use in my diagram are explained below:



Level 0 Dataflow Diagram

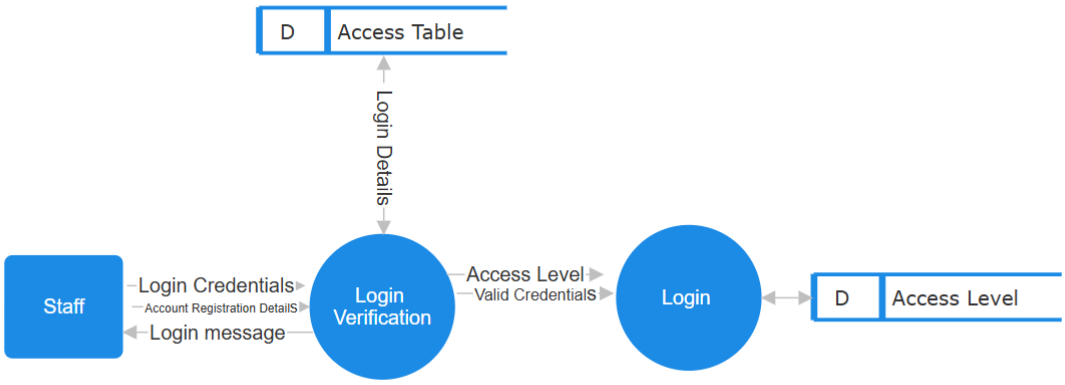
Below is a very simple level 0 data flow diagram for the system



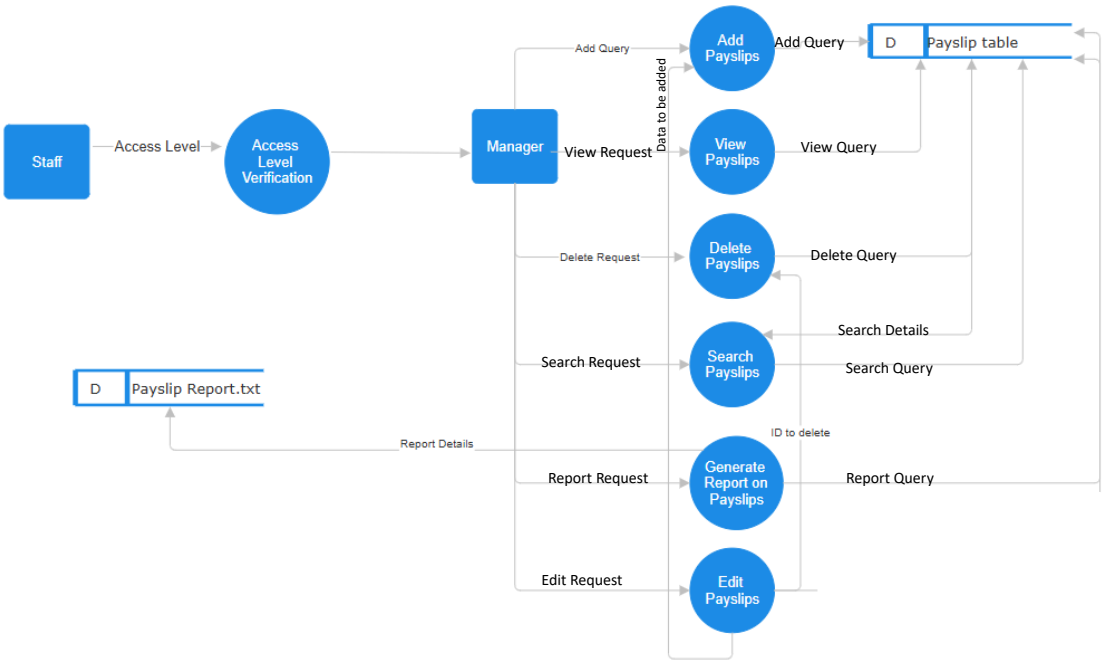
Level 1 Dataflow Diagrams

Below is a more detailed overview of the flow of data for some of the processes in my program.

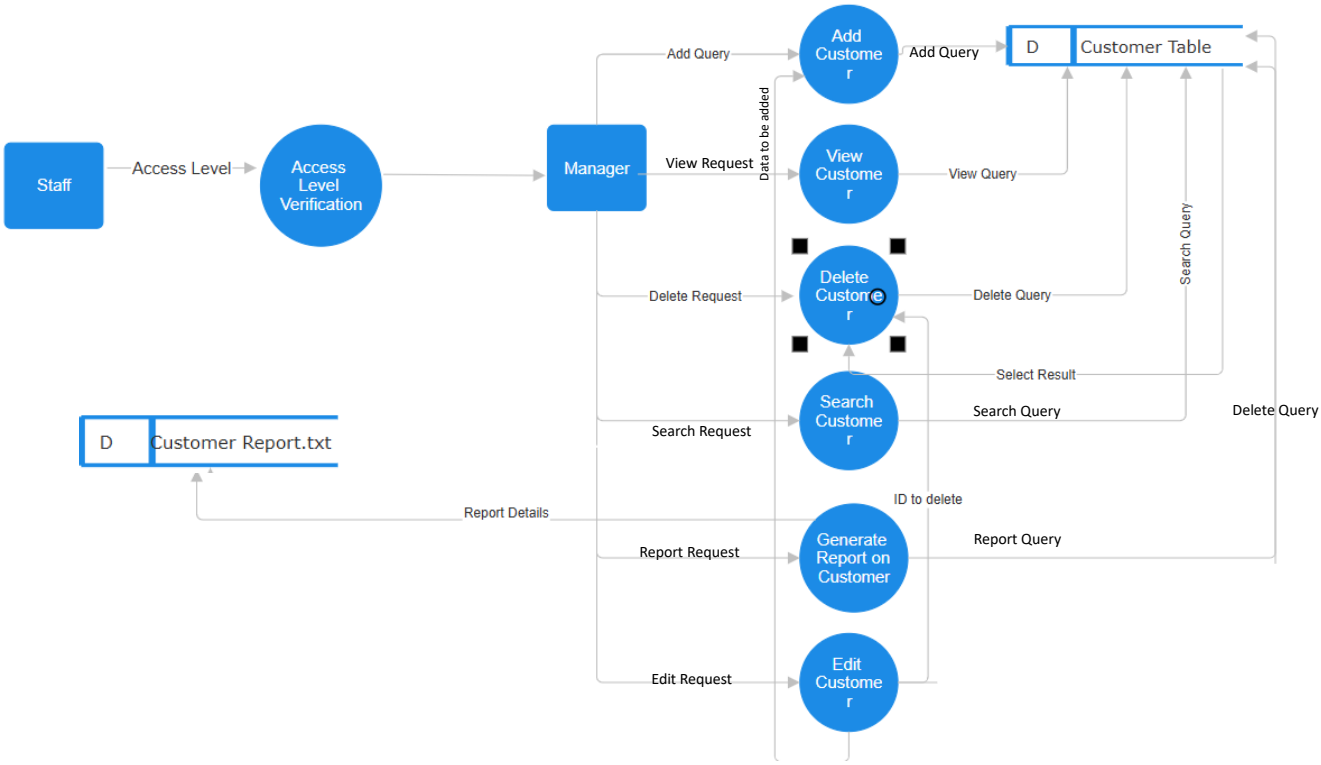
Login Dataflow



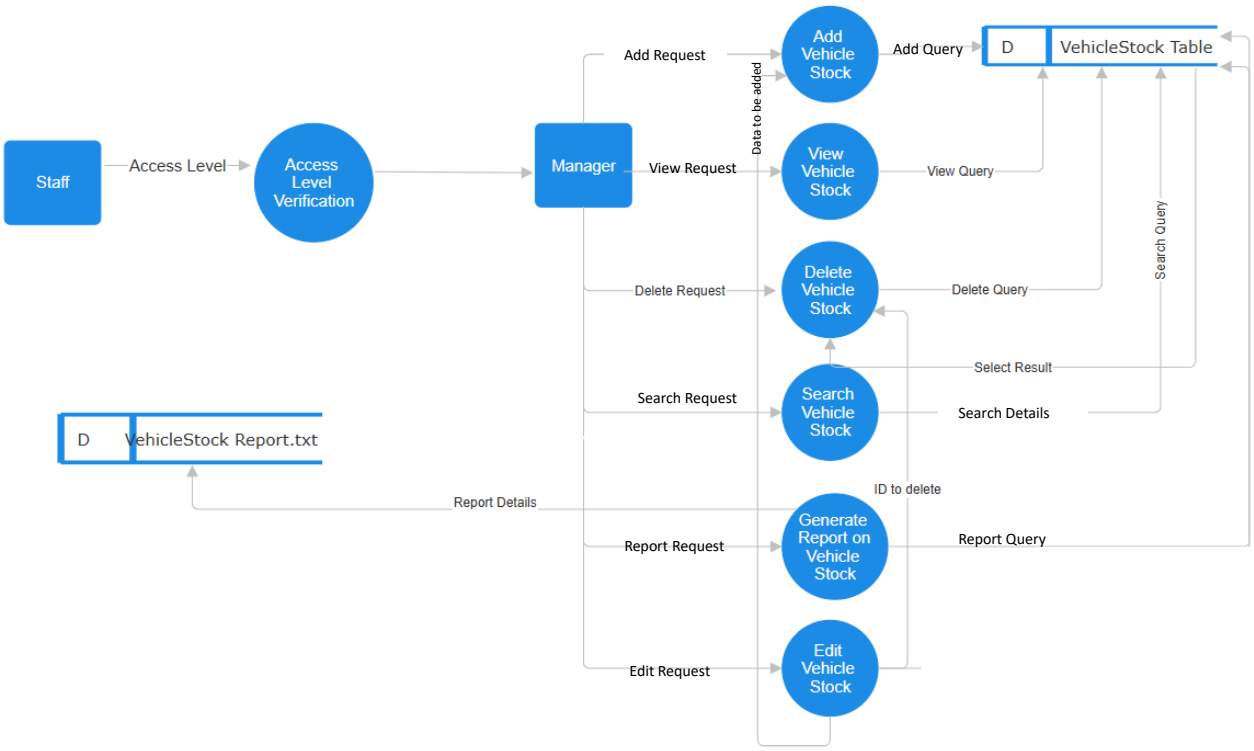
Payslips Dataflow



Customer Dataflow



Vehicle Stock



3.5 Graphical User Interface

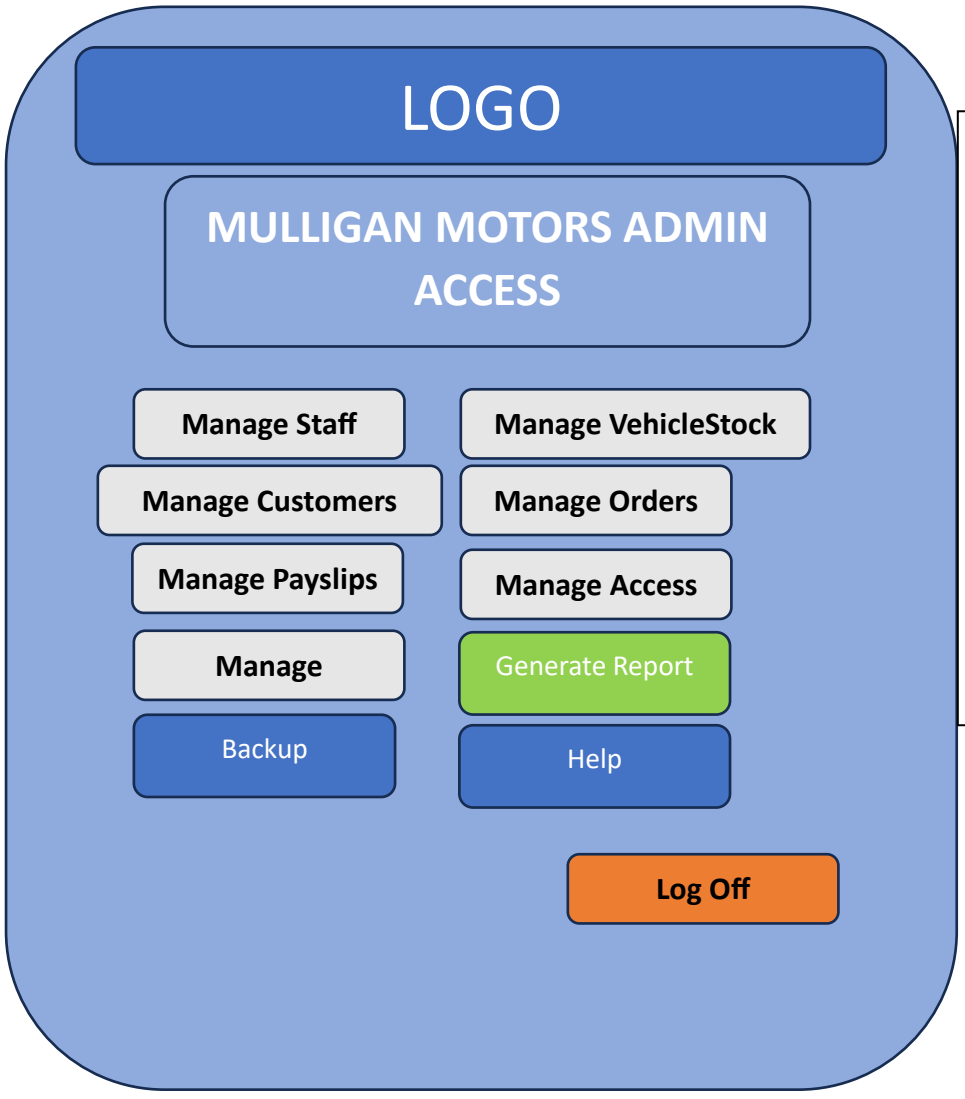
I plan to create an initial design for the GUIs to guide the development of the user interfaces later on during the development cycle. I believe that a well-organized and visually appealing interface will greatly enhance the user-friendliness of the solution. It's important to note that this is just a basic concept of how the GUIs should look, and the final product may differ significantly from the designs presented here.

LOGIN SCREEN



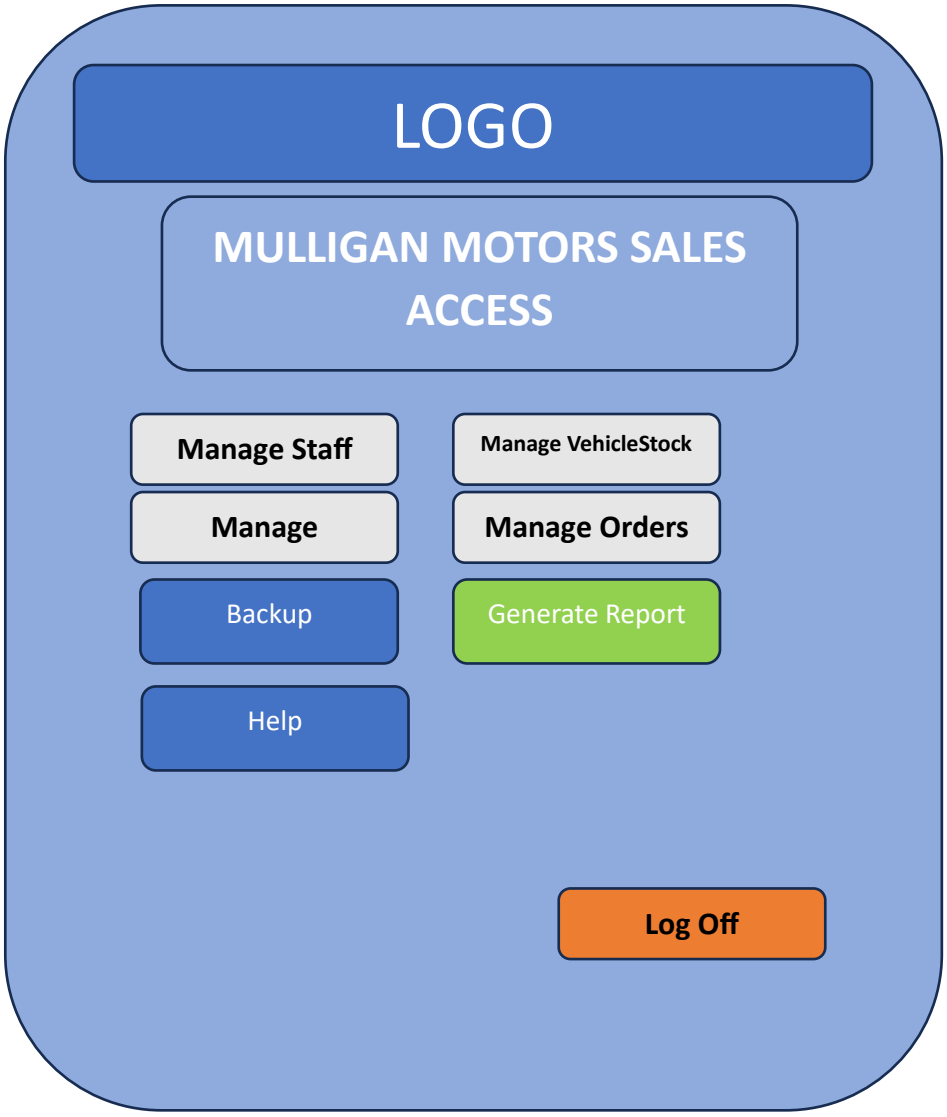
The login screen is the first form the users will see after executing the program. Depending on the login access of the username and password, the next form will either be an 'admin' or 'sales' main menu.

ADMIN ACCESS MAIN MENU



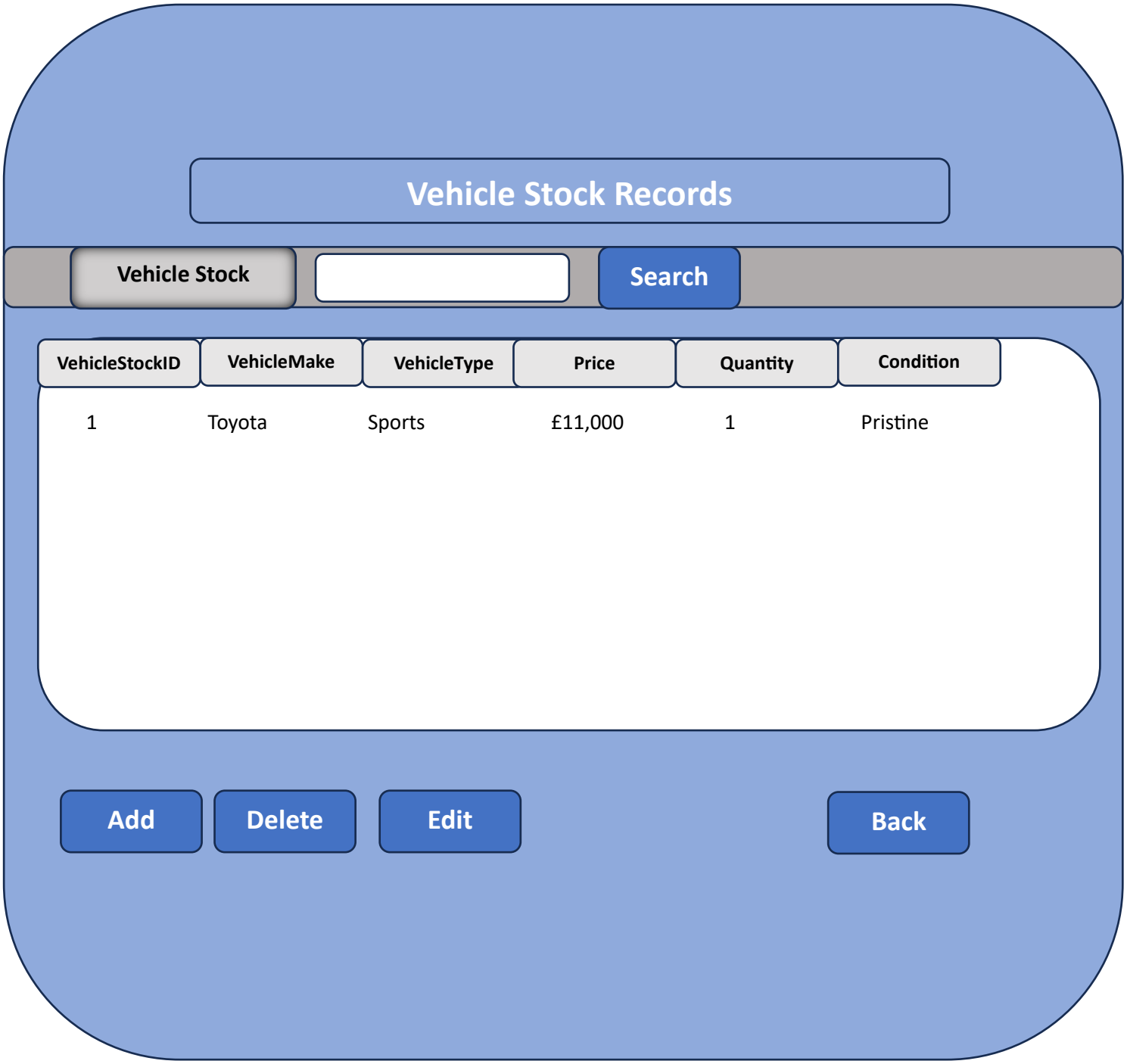
This is the main menu with the complete access to all actions in the program. The use of colours help distinguish each of my solutions planned function. The main menu is planned to act as a navigational hub to all sections of the software from navigating records to creating backups.

SALES ACCESS MAIN MENU



This is the main menu for those without the privileged 'admin' access, such as low-level sales employees. Resultingly, they are limited to the four basic tables. Despite this, they still have access to generating reports and backups ect.

MANAGE TABLE FORM



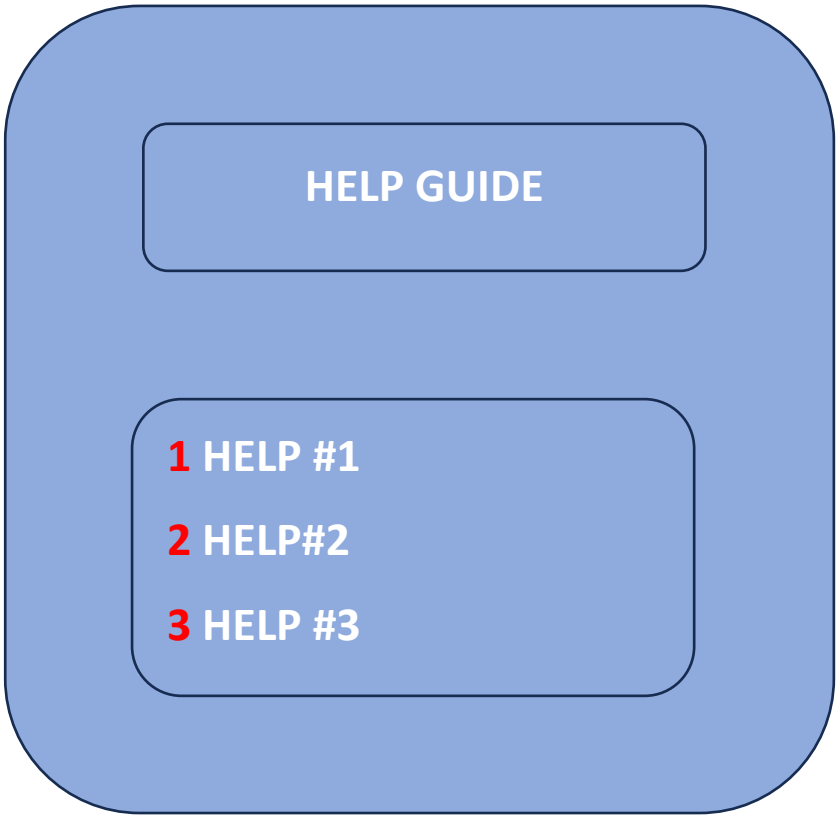
This is the planned form design for managing records and entities in the database system. I plan to utilise a tree view to showcase each record as seen. The example table I have used for this is tbl_VehicleStock however it should be noted it will be designed the exact same for each table. The entities attributes will be listed and the top with corresponding data aligned below with a focus on an organised and intuitive aesthetic. The user can clearly see the ADD, DELETE, and EDIT buttons which when pressed will begin their respective processes. The BACK button will navigate the user back to main menu. Finally, the search function is at the top of the form, and the user can select **any** attribute in the table via a drop down box to search for.

REPORT GENERATION



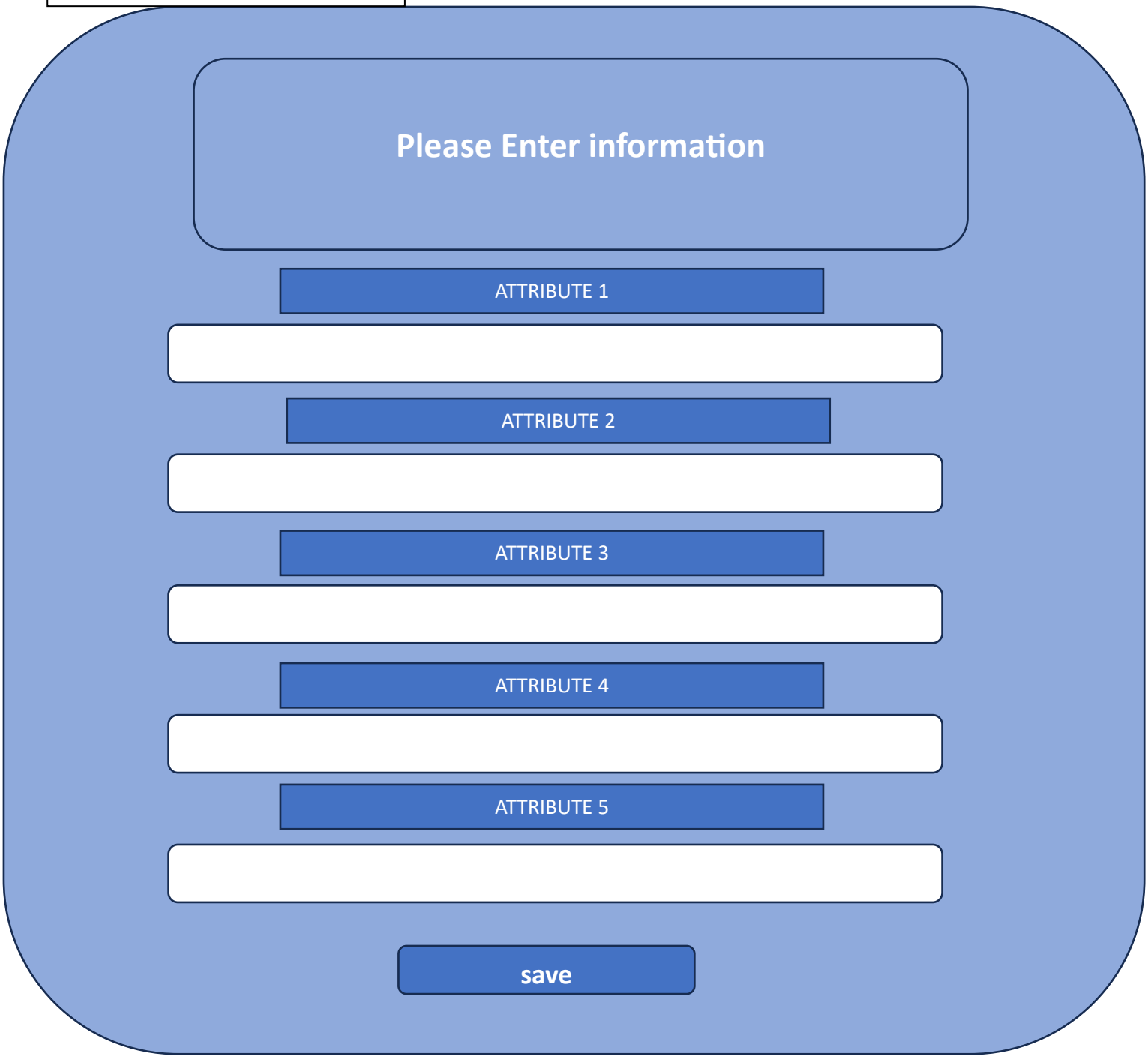
The report generation feature will be available from the main menu. The user will select a table from a drop down list and click generate report to complete the process.

HELP PAGE



The help menu will be accessible by clicking on the 'help' button in the main menu. This will contain a simple label detailing the features of my completed system with no interactive elements

INPUT FORM



This is how I plan to universally design every form that asks for the user's input, such as the add/edit processes, or account creation form. A label and text box pair design is simple and intuitive for users reducing Mulligan Motor's training cost. For example, the white space below attribute 1 is a box the user can enter information in to represent attribute 1.

At the end, the user can press the 'save' button to complete the process and have the system validate and store the inputted data.

These are my proposed designs for the data entry screens in the final program, along with the outputs generated by the output button on each screen. For these screens to function correctly, they must support both data input and output. Each screen will feature several labels and input boxes where users can enter data. The number of these elements will vary based on the number of rows in the corresponding database table, as some tables have more rows than others. Additionally, the screens must have the capability to print details for a specific record in the corresponding table. To add an item to the database, the user should open the input form and enter all necessary information into the appropriate input boxes and press the correctly labelled button. This action will trigger the corresponding subroutine and display a confirmation message indicating that the data has been successfully added to the database.

The delete function will only need the ID of the item to delete it. This is because all the data for the item will deleted, but only the unique ID is needed to find it. So, just the ID is needed to delete or find a record in the table. If the user wants to print the details of an item, the system will look for the vehicles ID. If the item can't be found, a message will let the user know. If the item is found, its details will be shown in a box on the screen. Also, if any required information is missing from the table, a message will notify the user. This is the same for searching, however, any attribute of an record can be used to find it.

3.6 Data dictionaries & Compulsory files

In addition to the code, two extra files are needed: a database (.db) file and a text (.txt) file.

The database file (named MulliganMotors.db) will store all the dealership's data. It will allow users to view, add, delete, edit, and search for data related to vehicles, staff, customers, and test drives. The program must be able to read (view/search) and write (add/delete/edit) to this file. The database should be organized well to a third normal form standard to make it faster to access and reduce duplicate and redundant data. This file will be accessed randomly, meaning it can quickly find any record. The database itself will also store the username and passwords for the login function as well as the access privileges these login credentials grant.

Text files will also be used to create reports from the database. There will be one text file for each table in the database (e.g., vehicles, staff, customers, test drives), formatted into a table for easy viewing. This makes it easier to share reports between users or to copy and edit as needed. Although you can view data within the program, sharing data this way would require sending the database file, which needs special software to open (especially on Windows 10), or writing out the results manually, which takes time and could lead to mistakes.

Data dictionaries explain the structure of database by displaying information such as data type, attributes, and validation checks applied to any fields. Each entity in my database solution will have a data dictionary to allow easy maintenance and complete understanding of the system for every function or table. They will be useful during the databases later

stages such as testing, as I can view what the database should be doing and compare it to its actual actions, and prototyping, as I can design the prototype system around the structure, I have laid out below.

Staff Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
StaffID	Primary key	Integer	Type, presence	Unique identifier for each staff member	1
Title		String	Type, presence	Title of the staff member (e.g., Mr., Mrs., Dr.)	Mr.
FirstName		String	Type, presence	First name of the staff member	John
LastName		String	Type, presence	Last name of the staff member	Doe
DateOfBirth		Date	Type, presence, format	Birth date of the staff member	1980-01-01
Position		String	Type, presence	Job title/role of the staff member	Salesperson
PhoneNumber		String	Type, presence, format	Contact phone number of the staff member	123-456-7890
AccessID		Integer	Type, presence	Foreign key linking to the Access table	2
PayslipID		Integer	Type, presence	Foreign key linking to the Payslip table	3

Payslip Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
PayslipID	Primary key	Integer	Type, presence	Unique identifier for each payslip	1

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
StaffID		Integer	Type, presence	Foreign key linking to the Staff table	1
HoursWorked		Real	Type, presence, range	Number of hours worked by the staff member	40.5
HourlyRate		Real	Type, presence, range	Hourly rate for the staff member	15.75
TotalOwed		Real	Type, presence, range	Total amount owed based on hours worked and rate	638.62

Access Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
AccessID	Primary key	Integer	Type, presence	Unique identifier for each access record	1
Username		String	Type, presence, format	Staff member's username for system login	jdoe
Password		String	Type, presence, format	Encrypted password for system security	********
AccessLevel		String	Type, presence	Defines the access rights (e.g., Admin, User)	Admin

VehicleStock Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
VehicleID	Primary key	Integer	Type, presence	Unique identifier for each vehicle in stock	1
VehicleName		String	Type, presence	Name of the vehicle	2019 Honda Civic
VehicleMake		String	VLookUp	Manufacturer of vehicle	Honda
VehiclePrice		Real	Type, presence, range	Price of the vehicle	15000.00
VehicleType		String	Type, presence	Type of the vehicle	Vehicle

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
ItemQuantity		Integer	Type, presence, range	Quantity of the item available in stock	5

Orders Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
OrderID	Primary key	Integer	Type, presence	Unique identifier for each order	1
ItemQuantity		Integer	Type, presence, range	Quantity of the item ordered	3
OrderValue		Real	Type, presence, range	Total value of the order	45000.00
DatePlaced		Date	Type, presence, format	Date when the order was placed	2024-01-15
DateArriving		Date	Type, presence, format	Date when the order is expected to arrive	2024-02-01
VehicleID		Integer	Type, presence	Foreign key linking to the VehicleStock table	1
CustomerID		Integer	Presence	Foreign key linking to the Customer table	1

Customer Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
CustomerID	Primary key	Integer	Type, presence	Unique identifier for each customer	1
FirstName		String	Type, presence	First name of the customer	Jane
LastName		String	Type, presence	Last name of the customer	Smith
EmailAddress		String	Type, presence, format	Email contact for the customer	jane.smith@email.com

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
PhoneNumber		String	Type, presence, format	Phone number for contacting the customer	123-456-7890
Address		String	Type, presence	Customer's home address	123 Main St, Springfield, IL
PreferredContactMethod		String	Type, presence	Email, phone, or other communication preference	Email

Test Drive Table (Access Method = Random)

Field Name	Keyfield	Data Type	Method of Validation	Description	Example
TestDriveID	Primary key	Integer	Type, presence	Unique identifier for each test drive booking	1
CustomerID		Integer	Type, presence	Foreign key linking to the Customer table	1
VehicleID		Integer	Type, presence	Foreign key linking to the VehicleStock table	1
TestDriveDate		Date	Type, presence, format	Date when the test drive is scheduled	2024-01-20
TestDriveTime		Time	Type, presence, format	Time when the test drive is scheduled	10:00 AM
Duration		Integer	Type, presence, range	Duration of the test drive (in minutes)	30
Status		String	Type, presence	Status of the test drive (e.g., Pending, Completed)	Pending
SalespersonID		Integer	Type, presence	Foreign key linking to the Staff table	1
Feedback		String	Type	Customer feedback after the test drive (optional)	Great car!

Report File:

	MULLIGAN	*MOTORS*	*(LOGO)*
Heading 1	Heading 2	Heading 3	Heading 4
ID1	FIELD1	FIELD1.2	FIELD1.3
ID2	FIELD2	FIELD2.1	FIELD2.2
ID3	FIELD3	FIELD3.1	FIELD3.2
ID4	FIELD4	FIELD4.1	FIELD4.2
ID5	FIELD5	FIELD5.1	FIELD5.2
ID6	FIELD6	FIELD6.1	FIELD6.2

These data dictionaries I have created should help me, or someone else, in the development and maintenance of this digital database solution. I will refer to these often when creating the database entities and general table structure to ensure that everything is correct. Additionally, this will be a huge help when testing the system as I can see what should be included in the system and test it to see if that's the case.

3.8 Normalisation

Non-Normalized Form (Unnormalized Data)

All the data is stored in a single table. This table is prone to data redundancy and anomalies.

(StockID, Name, Price, StockType, StockLevel, OrderID, ItemName, Quantity, OrderValue, DatePlaced, DateArriving, StaffID, Title, FirstName, LastName, PhoneNumber, DateOfBirth, PayslipID, HoursWorked, HourlyRate, AccessID, Username, Password, AccessLevel, TotalOwed, TestDriveID, CustomerName, CustomerContact, TestDriveDate, TestDriveTime, VehicleID, StaffID, ServiceID, ServiceType, ServiceDate, VehicleID, CustomerID, SalesID,

VehicleID, CustomerID, SaleDate, SalePrice, CustomerID, CustomerName, CustomerContact, CustomerEmail)

First Normal Form (1NF)

The table is organized such that each column contains atomic values, and each column contains only one value for each record.

Staff(StaffID, Title, FirstName, LastName, PhoneNumber, Position, DateOfBirth, HoursWorked, HourlyRate, TotalOwed, Username, Password, AccessLevel)

Stock(StockID, Name, Price, StockType, StockLevel)

Orders(OrderID, ItemName, Quantity, OrderValue, DatePlaced, DateArriving, StockID)

TestDrive(TestDriveID, CustomerName, CustomerContact, TestDriveDate, TestDriveTime, VehicleID, StaffID)

Service(ServiceID, ServiceType, ServiceDate, VehicleID, CustomerID)

VehicleSales(SalesID, VehicleID, CustomerID, SaleDate, SalePrice)

Customer(CustomerID, CustomerName, CustomerContact, CustomerEmail)

Second Normal Form (2NF)

In addition to 1NF, all non-key attributes are fully functional dependent on the primary key. In this case, composite keys (if any) are removed and redundancy is minimized.

Staff(StaffID, Title, FirstName, LastName, Position, PhoneNumber, DateOfBirth, PayslipID, AccessID)

Payslip(PayslipID, HoursWorked, HourlyRate, TotalOwed)

Access(AccessID, Username, Password, AccessLevel,)

Stock(StockID, Name, Price, StockType, StockLevel)

Orders(OrderID, ItemName, Quantity, OrderValue, DatePlaced, DateArriving, StockID)

TestDrive(TestDriveID, CustomerName, CustomerContact, TestDriveDate, TestDriveTime, VehicleID, StaffID)

Customer(CustomerID, CustomerName, CustomerContact, CustomerEmail)

Third Normal Form (3NF)

In addition to 2NF, all the attributes are functionally dependent only on the primary key. There are no transitive dependencies.

Staff(StaffID, Title, FirstName, LastName, Position, PhoneNumber, DateOfBirth, PayslipID,)

Payslip(PayslipID, HoursWorked, HourlyRate, TotalOwed, StaffID)

Access(AccessID, StaffID Username, Password, AccessLevel)

VehicleStock(VehicleID, Name, Price, VehicleType, StockLevel)

Orders(OrderID, ItemName, Quantity, OrderValue, DatePlaced, DateArriving, VehicleID, CustomerID)

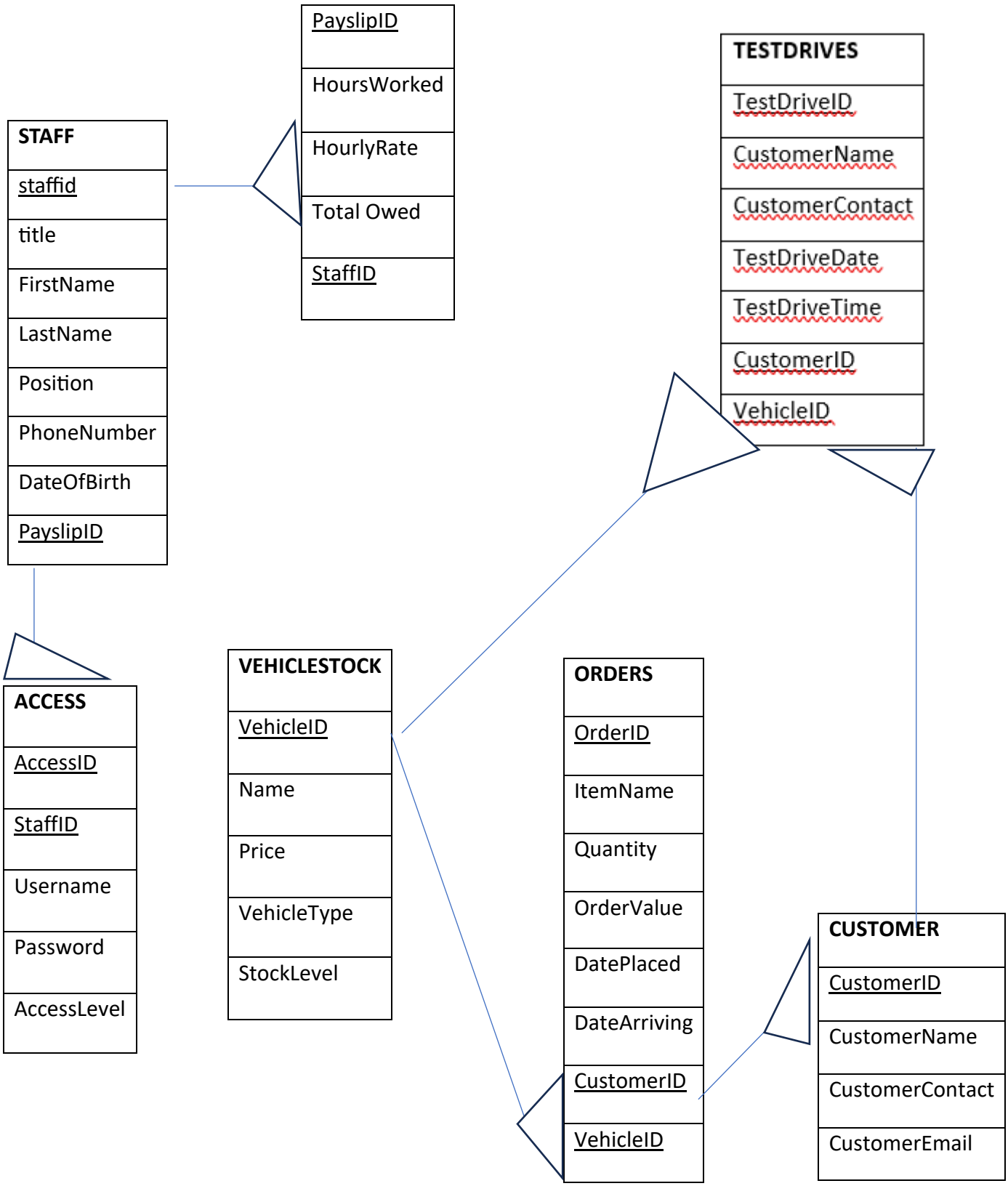
TestDrive(TestDriveID, CustomerName, CustomerContact, TestDriveDate, TestDriveTime, VehicleID, CustomerID)

Customer(CustomerID, CustomerName, CustomerContact, CustomerEmail)

This structure ensures that the data is stored efficiently, with minimal redundancy and maximized integrity. Each table represents a specific entity and includes only attributes related to that entity.

3.9 Entity relationships




PayslipID
HoursWorked
HourlyRate
Total Owed
StaffID
STAFF
staffid
title
FirstName
LastName
Position
PhoneNumber
DateOfBirth
PayslipID
TESTDRIVES
TestDriveID
CustomerName
CustomerContact
TestDriveDate
TestDriveTime
CustomerID
VehicleID
ACCESS
AccessID
StaffID
Username
Password
AccessLevel
VEHICLESTOCK
VehicleID
Name
Price
VehicleType
StockLevel
ORDERS
OrderID
ItemName
Quantity
OrderValue
DatePlaced
DateArriving
CustomerID
VehicleID
CUSTOMER
CustomerID
CustomerName
CustomerContact
CustomerEmail

3.10.0 Pseudocode

The final design method I will use is pseudocode. Pseudocode is a way of defining an algorithm that does not adhere to the syntax of any specific programming language. Instead, it closely resembles programming language syntax, making it easy to translate into the developer's chosen language (in my case, Python). To enhance readability and minimize repetition, only a few examples will be provided. Since much of the code in the final program will likely be very similar, extensive examples are unnecessary.

3.10.1 Validation Pseudocode

Data validation is essential for the final product, as outlined in the data dictionary. This process involves verifying that all data entered into the system adheres to predefined rules within the program. By doing so, it ensures that the data is accurate and consistent, significantly reducing the likelihood of human error during data entry. This validation helps maintain data integrity and reliability across the system.

Presence Check

The presence check aims to ensure that data has been entered into the system. It works by verifying if the input field is empty. If it is, the user receives an error message prompting them to enter the required data. If data is present, the user receives a confirmation that the information has been successfully entered into the system.

```
1       Input StaffID
2       If StaffID == “”
3       Output (“Do not leave the field blank, record not
        saved”)
4       Else
5       Output(“StaffID validated, Input will be saved”)
6       END
7
8
9
```

Type Check

The purpose of the type check is to ensure that the data entered is of the correct data type. This check validates that the input matches the expected type (e.g., number, text, date). If the data type is incorrect, the user receives an error message indicating the required data type. If the data type is correct, the user receives confirmation that the data has been successfully entered into the system. I achieved this with a try and except command in python.

```
Define StaffID as integer
Try:
Input StaffId
Except ValueError:
Output ("Please enter StaffID as an integer")
Else
Output ("Input validated")
END

```

Format Check

The purpose of the format check is to ensure that the data entered follows a specific pattern or format. This check validates that the input matches the expected format, such as a phone number, email address, or date format. If the data does not conform to the required format, the user receives an error message specifying the correct format. If the data is in the correct format, the user receives confirmation that the data has been successfully entered into the system. I found this simple format check online and decided to include it in my software.

```
Input CustomerEmail
Def isValidEmail(CustomerEmail)
Return '@' in email

If isValidEmail(CustomerEmail):
Output ("Email is validated")
Else
Output ("Email is not correct format, it should
contain a @")
END
```

Length Check

The purpose of the length check is to ensure that the data entered meets the required length constraints. This check verifies that the input is neither too short nor too long. If the data does not meet the length requirements, the user receives an error message indicating the acceptable length. If the data length is correct, the user receives confirmation that the data has been successfully entered into the system.

```
Input StaffName
If Length of StaffName > 25 OR < 2 then
Output ("Name either too long or short, Input not saved")
Else
Output ("Name validated, Input saved")
END
```

Range Check

The purpose of the range check is to ensure that the data entered falls within a specified range of values. This check verifies that the input is between predefined minimum and maximum limits. If the data is outside the acceptable range, the user receives an error message indicating the allowed range. If the data is within the range, the user receives confirmation that the data has been successfully entered into the system.

```
Input StaffAge
If staffage < 16 OR > 90
Output ("Staff age is out of bounds, input not saved")
Else
Output ("Age validated, record saved")
END
```

3.10.2 Processes Pseudocode

Adding a record:

```
1       Connect to db
2       Conn cursor
3       When SaveRecordButton == clicked:
4       Values = entires.get()
5       If Validate(Values) == True:
6       Cursor(INSERT INTO staff, VALUES(Values))
7       else
8       Output ("Validation for this record has failed")
9       END
```

To add records, a first function will get all the inputs from the GUI text boxes that are required to create a new record, and a second function will validate them. If both are successful, the system will use SQL to insert the values into the table and save them. Otherwise, an error window will appear

Deleting a record:

```
1       Connect to db
2       Conn cursor
3       Input ID
4       When DeleteButton == Clicked:
5       Cursor( DELETE FROM StockVehicle WHERE VehicleID = ID
6       Cursor.Commit
7       Cursor.close
8       END
```

To delete a record I have showcased the SQL code that will be required. The user will input the unique identifier of a record, the primary key, to point out the record to be deleted. Once the delete button is pressed in the GUI, SQL code will run deleting the record with the matching ID from the table.

Editing a record

```
1       Connect to db
2       Conn cursor
3       Input ID
4       Delete record():
5       Add Record():
6       END
```

To edit a record, I have made functions out of adding and deleting a record and reused them in combination to edit data. This will reduce the size of the code and if I decide to make adding or deleting a file more efficient, I will only need to edit it in one place (where the function is). This should result in easier maintenance and quicker file performance.

Logging into the system:

```
1       Connect to db
2       Conn cursor
3       Input username
4       Input password
5       If username OR password == " " then
6       Ouput "Field must not be left blank"
7       Else
8       Search username AND Password in login.txt
9       If username AND password in login.txt then
10      Output("Credentials verified")
11      Allow access
12      Else
13      Output ("Credentials incorrect")
14      Deny access
15      END
```

In my database system, I plan to store login credentials in a text file. To login, the user will open the login window and input a username and password. To ensure that the field is not left blank, I will also run a presence check on the two fields. Otherwise, the system will compare the username and password to the ones present in the text (.txt) file. If they match, the user will be granted access, if they don't, an error window will pop up highlighting the incorrect credentials and the user will need to try again.

Searching the system:

```
1       Connect to db
2       Conn cursor
3
4       SearchItems = list
5       SearchItems = Entries.get()
6       When searchbutton == Clicked:
7       Rows = SELECT * FROM VehicleStock
8       For row in Rows
9       ListRows = list(Rows)
```

```
10      If SearchItems[x]== “”
11      ListRows[x] = “”
12      If SearchItems = ListRows:
13      ids.append(ListRows[0])
14      Output ids
15      END
```

To search for a record in the system, the user will enter any values they want to search for a text box. These values will then be compared against all the records in a given table (in this example it is tbl_VehicleStock). If any of the terms in the Search Items appear in the record, the record will be saved and be outputted at the end of the algorithm.

Outputting a record:

```
1       Connect to db
2       Conn cursor
3       Input ID
4       When outputButton == Clicked:
5       Report = SELECT * FROM staff WHERE staffid = ID
6       Output Report
7       END
```

To output a record, assumingly in the form of a report, the user must press the output button and input an ID. After which, an SQL query will run that will collect the record with a matching ID. This will display the entirety of the record with all it’s attributes in a special format made to be aesthetically pleasing.

Check access level for a file:

```
1       Connect to db
2       Conn.cursor
3       When ViewStaffButton == clicked:
4       If LoginAccessLevel == “Admin”:
5       Allow access
6       Else
7       Output (“You do not have the required admin
        permissions to access this file”)
8       Deny Access
9       END
```

Booking a test drive:

```
1       Connect to db
2       Conn.Cursor
3       when SaveButton == clicked then:
4       Validate(inputs)
5       If validation == True then:
6       For record in TestDrives:
7       If inputs == record in TestDrives then:
8       Output "This car is already taken at this time"
9       Else
10      Add record(values)
11      Output ("Test drive booked successfully")
12      END
```

To book at test drive, you must enter all relevant information such as vehicle, date, and time. After pressing the save button, the inputs will be validated and if they pass the system will then compare the inputs against records also in tbl_TestDrives. To avoid conflicting reservations/double booking, if the inputs are the same as a record in the table, it will not save and the user will be given an error message. Else, the records will be saved and an output window will inform the user that the test drive was booked successfully.